

MLP pour MNIST et moteur de différentiation automatique

Extensions S2 : GEMM, CNN, Tiny-ImageNet et MPI

Jianye Shi Hao Qian Xiang Bian Abdenmour Boulmis

Master CHPS – UVSQ
Encadrement : Aurélien Delval

6 mai 2026

1. Introduction

- ▶ Point de départ : un prototype MNIST avec un MLP, de l'autodiff en mode reverse et une boucle d'entraînement.
- ▶ Objectif du semestre : élargir ce prototype vers un système expérimental plus complet.
- ▶ Trois directions :
 - ▶ GEMM et convolution pour le calcul bas niveau ;
 - ▶ modèles et données plus exigeants, avec CNN (puis ResNet) et Tiny-ImageNet ;
 - ▶ entraînement distribué avec MPI, puis chemin inspiré d'IST-DP.

Idée centrale

Le même runtime d'entraînement doit rester réutilisable quand on change à la fois d'opérateurs, de modèles, de données et de mode d'exécution.

2. GEMM : méthodologie et environnement

Pourquoi commencer par GEMM ?

- ▶ GEMM est l'opérateur central des couches fully connected.
- ▶ Le chemin convolutif im2col + GEMM dépend aussi directement de sa performance.

Implémentations comparées

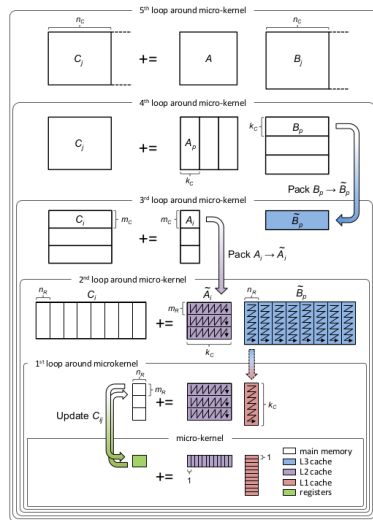
- ▶ blas, omp, omp_blocked, omp_blocked_packb, omp_blocked_packab, omp_gotoblas_avx2, omp_gotoblas_avx512
- ▶ Les workloads viennent de formes réelles du projet : couches MLP et CNN, pas uniquement des matrices synthétiques.

Protocole / environnement

- ▶ Plateforme contrôlée : Ryzen 9 8940HX, fréquence fixée, OpenBLAS + OpenMP
- ▶ Comparaisons limitées à 1–8 threads physiques, avec OMP_PROC_BIND=true et OMP_PLACES=cores
- ▶ Restriction expérimentale à 8 CPU physiques via taskset -c 0–7
- ▶ 5 warm-up, puis 2000 fois répétées avec médiane du temps
- ▶ SMT désactivé : sur GEMM, les threads logiques se partagent les mêmes unités d'exécution, avec peu de gain et plus d'interférences.

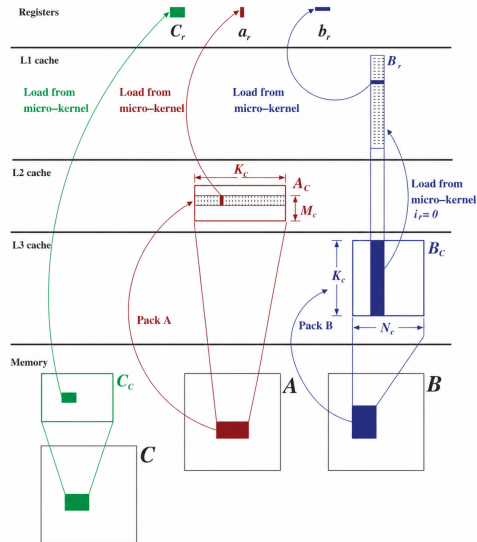
2. GEMM : définitions des implémentations

- blas : appel à OpenBLAS, utilisé comme référence externe.
- omp : version OpenMP de base, sans blocking structuré ni packing.
- omp_blocked : ajout d'un blocking cache pour améliorer la réutilisation des sous-blocs (BlockSize=64).
- omp_blocked_packb : blocking + packing du panneau B seulement (BlockSize=48).
- omp_blocked_packab : blocking + packing explicite des panneaux de A et B (PackM=32, PackN=128, PackK=16).
- omp_gotoblas_avx2 : organisation de type GotoBLAS avec micro-kernel AVX2 en float32 (`_mm256_loadu_ps`, broadcast, FMA).
- omp_gotoblas_avx512 : même principe général, avec micro-kernel AVX-512 (par défaut `avx512_8x32`), unroll $\times 4$ de la boucle K (directive compilateur, pas déroulage manuel complet) et prefetch explicite des données packées.

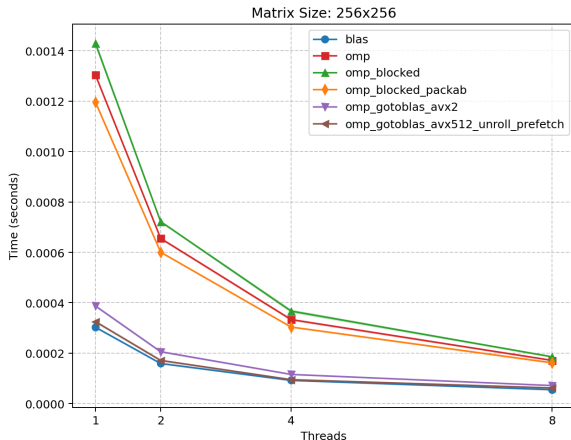


2. GEMM : organisation GotoBLAS

- Schéma conceptuel : C_r dans les registres ; A_r/B_r au plus près du micro-kernel ; A_c plutôt orienté cache privé, B_c plutôt orienté cache partagé.
- Le packing régularise les accès. Cette lecture guide les paramètres, sans supposer une résidence stricte et permanente de chaque bloc dans un niveau de cache donné.
- Configuration fixe retenue : avx2_8x8, $K_c = 384$, kCacheBlockM= 8 (soit M_c effectif = 64), $N_c = 448$. Le détail du réglage figure en annexe.



2. GEMM : résultats de montée en charge (256x256)



- ▶ Ici on fixe une seule taille synthétique, 256×256 , et on compare les temps de 1 à 8 threads.
- ▶ Le blocking et le packing améliorent déjà nettement la base omp.
- ▶ Les chemins `omp_gotoblas_avx2` et la variante AVX-512 descendent plus vite avec le nombre de threads.
- ▶ À 8 threads sur ce cas, OpenBLAS reste le plus rapide, mais l'écart avec les variantes GotoBLAS devient beaucoup plus faible.

3. Optimiseurs : SGD, MomentumSGD et AdamW

Les trois variantes modifient la même information de base : le gradient courant.

$$\text{SGD: } \theta_{t+1} = \theta_t - \eta g_t$$

$$\text{Momentum: } v_{t+1} = \mu v_t - \eta g_t, \quad \theta_{t+1} = \theta_t + v_{t+1}$$

$$\text{AdamW: } \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t$$

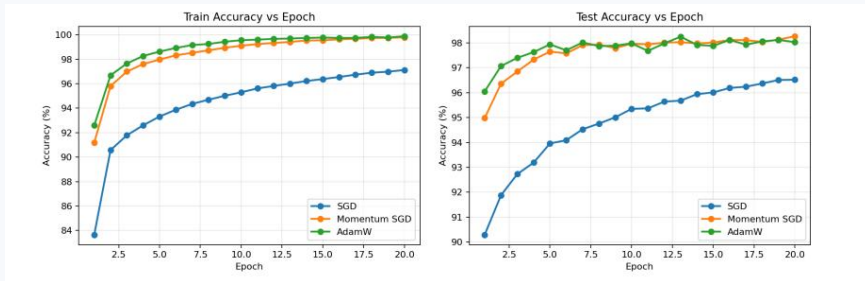
SGD converge plus lentement et reste en retrait.

MomentumSGD atteint la meilleure précision test : 98,27%.

AdamW est très proche : 98,03%, avec un démarrage rapide.

3. Boucle d'entraînement commune

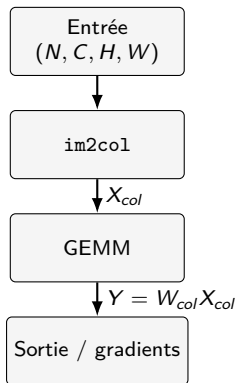
Trainer::runEpoch(...) regroupe apprentissage et évaluation pour MLP, CNN, ResNet et modes distribués.



Comparaison des précisions d'entraînement/test pour trois optimiseurs sur le MLP

AdamW converge rapidement grâce à son learning rate adaptatif et reste très utilisé dans les réseaux modernes.

4. Opérateur de convolution : pipeline



- ▶ Le backend convolutif implémente la chaîne $\text{im2col} \rightarrow \text{GEMM} \rightarrow \text{reshape}$.
- ▶ Cette factorisation permet de réutiliser un noyau GEMM déjà optimisé.
- ▶ En rétropropagation, col2im reconstruit le gradient par rapport à l'entrée.
- ▶ Le coût d'exécution ne se limite pas à GEMM :
 - ▶ transformations de données
 - ▶ mouvements mémoire

4. Opérateur de convolution : optimisations

- ▶ **Parallélisation**

- ▶ Découpage par sorties indépendantes
- ▶ Réduction des conflits d'écriture et du coût de synchronisation

- ▶ **Accès mémoire**

- ▶ Réorganisation des boucles `im2col/col2im`
- ▶ Accès plus continu, meilleure localité cache, latence plus stable

- ▶ **Buffers temporaires**

- ▶ Réutilisation des buffers intermédiaires
- ▶ Diminution des allocations/libérations répétées

- ▶ **Observation expérimentale**

- ▶ Gains plus marqués pour des charges moyennes à élevées
- ▶ Gains limités pour de petites charges (coût fixe du parallélisme)

5. MaxPooling : réduction spatiale et extraction des caractéristiques

Exemple

Entrée :

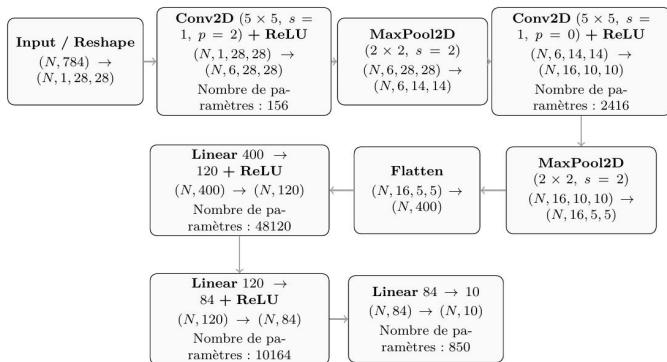
$$\begin{bmatrix} 1 & 2 \\ 5 & 3 \end{bmatrix} \rightarrow 5$$

Réduit la taille des cartes de caractéristiques

Diminue le coût de calcul des couches suivantes

5. Structure et flux de données du CNN actuel

Chemin inspiré de LeNet : deux convolutions, deux pooling, puis une tête entièrement connectée.

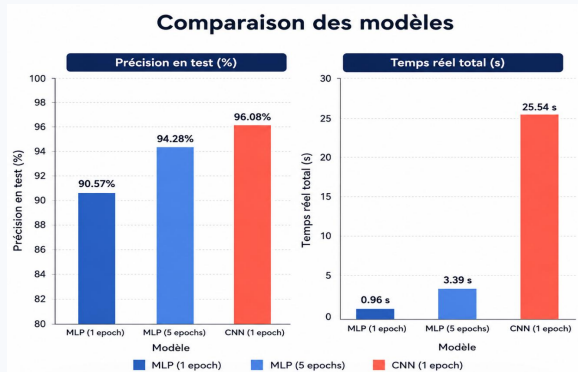


Nombre total de paramètres entraîables : 61706

évolution des shapes et nombre de paramètres : 61 706 paramètres entraîables.

5. Vérification à l'exécution : précision vs coût

Le CNN valide le chemin convolutif, mais son coût est beaucoup plus élevé que celui du MLP.



$$\text{accuracy} = \frac{\# \text{prédictions correctes}}{\# \text{exemples}}$$

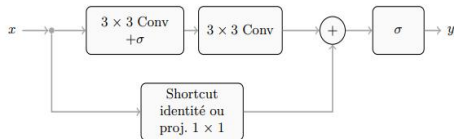
$$\text{ratio de coût} = \frac{T_{CNN,1e}}{T_{MLP,1e}} = \frac{25.54}{0.96} \approx 26.6 \times$$

$$\frac{T_{CNN,1e}}{T_{MLP,5e}} = \frac{25.54}{3.39} \approx 7.5 \times$$

Cette vérification sert de base avant ResNet et Tiny-ImageNet.

5. Extension vers une structure ResNet

La connexion résiduelle apprend une correction plutôt qu'un mapping complet.



$$H(x) = x + F(x), \quad y = F(x) + P(x)$$

Stages : 16 -> 32 -> 64 canaux ;

résolution $28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$.

Intérêt : réseau plus profond, optimisation plus stable.

6. Optuna : recherche automatique d'hyperparamètres

Rôle : remplacer une exploration manuelle coûteuse par des trials pilotés et comparables.

$$\text{score} = \begin{cases} \max_e \text{test_acc}_e, & \text{metric} = \text{best} \\ \text{test_acc}_E, & \text{metric} = \text{last} \end{cases} \quad \text{max score}$$

Espace exploré : learning rate, batch size,
activation, initialisation et optimiseur.

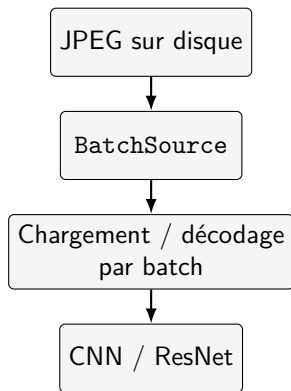
40 trials × 3 epochs

best test_acc = 0,9891

Gain observé

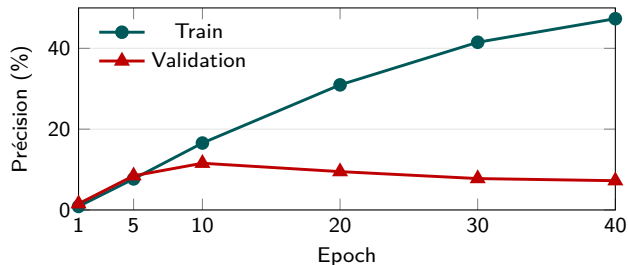
+0,46 point vs défaut manuel

7. Tiny-ImageNet : montée en complexité



- ▶ Tiny-ImageNet : 200 classes, images RGB 64×64 (3 canaux).
- ▶ Entraînement : $\sim 100\,000$ images (environ 500 / classe).
- ▶ Validation : 10 000 images (50 / classe).
- ▶ Par rapport à MNIST : plus de classes, données couleur, variabilité visuelle plus forte.
- ▶ Objectif de cette étape : valider le passage à l'échelle du pipeline C++.

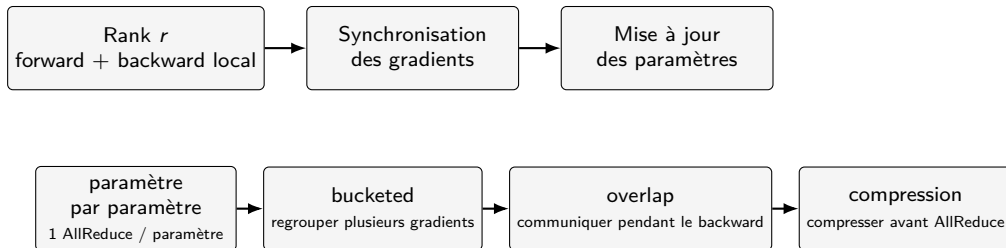
7. Tiny-ImageNet : résultat d'ingénierie



- ▶ Le pipeline Tiny-ImageNet est opérationnel dans le framework.
- ▶ La boucle d'entraînement reste la même (chargement, labels, optimisation).
- ▶ Les résultats actuels montrent un écart train/validation marqué.
- ▶ Le point principal ici est la faisabilité système, pas l'accuracy finale.

Lecture. L'étape valide l'intégration Tiny-ImageNet ; l'amélioration de la précision et du coût d'entraînement relève de la phase suivante (architecture, régularisation, tuning).

8. Entraînement distribué : organisation

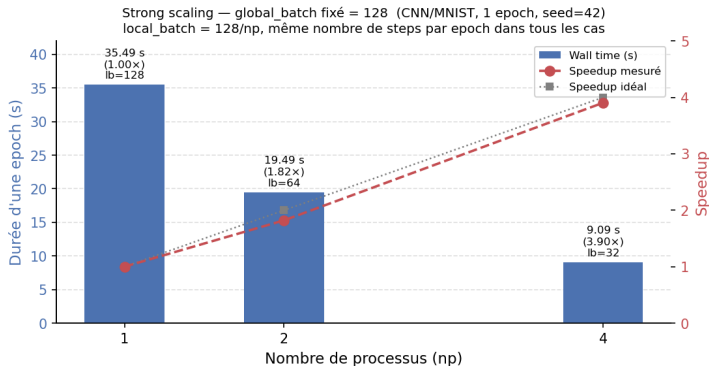


- Le schéma mathématique ne change pas : on reste en data parallel synchrone,

$$g_{\text{global}} = \frac{1}{B_{\text{global}}} \sum_r g_r$$

- bucketed réduit le nombre d'appels collectifs; overlap cherche à masquer sync_wait; compression vise à réduire le payload.

8. Entraînement distribué : résultats



Strong scaling : *global_batch=128* fixé, *local_batch=128/np*, *seed=42*.
Speedup : 1.00x → 1.82x → 3.90x

- **Strong scaling** (*global_batch=128* fixé) : même nombre de steps (≈ 469) pour tout np ; speedup 3.90x à $np=4$ — quasi-idéal car MPI en mémoire partagée (machine unique), pas sur réseau.

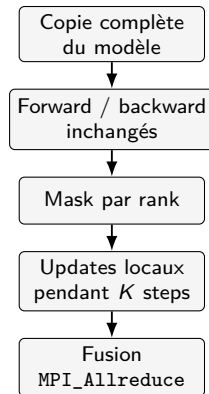
9. IST : motivation et principe

Point de départ

- ▶ Ce n'est pas l'IST complet de l'article.
- ▶ En data parallel classique : synchronisation à chaque step.
- ▶ Quand le nombre de workers augmente, cette synchronisation devient coûteuse.

Idée

Synchroniser moins souvent, sans changer toute la boucle d'entraînement.



Deux notions à retenir

- ▶ **Mask** : paramètres modifiables par un rank.
- ▶ **K** : nombre de steps locaux avant synchronisation.

9. IST : fenêtre locale et validation

Pendant la fenêtre locale

- ▶ chaque rank ne met à jour que sa partie des paramètres
- ▶ si K augmente : moins de synchronisations
- ▶ risque : modèles locaux trop différents si la fenêtre est trop longue
- ▶ fin de fenêtre : fusion des changements via `MPI_Allreduce`

Limite à rappeler

- ▶ communication encore dense
- ▶ gain surtout lié à la baisse de fréquence de synchronisation

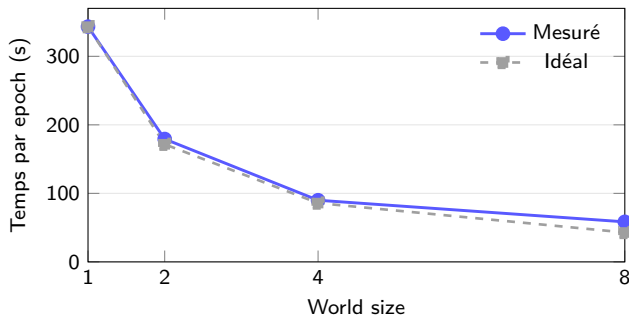
Expériences

- ▶ tests sur le cluster `zen.univ-lille.fr`
- ▶ plusieurs ranks MPI
- ▶ objectif : vérifier le fonctionnement distribué réel
- ▶ ablation sur K : meilleur compromis observé avec $K = 4$

Mode	Acc. val.	Temps
dense	3.51%	243 s
IST $K = 1$	3.51%	243 s
IST $K = 4$	4.33%	244 s

*Tiny-ImageNet, seed=42, 1 epoch,
world_size=2.*

9. Chemin inspiré d'IST : strong scaling



Speedup : $1.00\times \rightarrow 1.91\times \rightarrow 3.82\times \rightarrow 5.88\times$

Protocole

- ▶ Tiny-ImageNet, CNN
- ▶ IST avec $K = 4$
- ▶ `global_batch=256` fixé
- ▶ `local_batch=256/world_size`

Lecture

- ▶ temps : 343 s $\rightarrow$ 58 s
- ▶ scaling proche de l'idéal jusqu'à 4 workers
- ▶ à 8 workers, le temps baisse encore mais l'efficacité diminue

Conclusion. Le chemin IST fonctionne avec MPI sur cluster et montre un potentiel de passage à l'échelle.

Bilan

- ▶ Le backend GEMM a été clarifié, comparé et stabilisé pour servir de base aux expériences suivantes.
- ▶ Le projet dépasse désormais le seul cas MLP/MNIST : CNN, ResNet, Tiny-ImageNet et MPI sont intégrés dans un même runtime.
- ▶ Les chemins distribués couvrent déjà bucketed, un overlap_bucketed correctness-only, la compression côté runtime et une variante inspirée d'IST encore expérimentale.
- ▶ Les prochaines priorités sont claires : convolution plus efficace, pipeline d'entrée plus riche, meilleure généralisation sur Tiny-ImageNet et communication plus creuse côté IST.

Questions

Annexe : configuration cluster

Nœud et pile

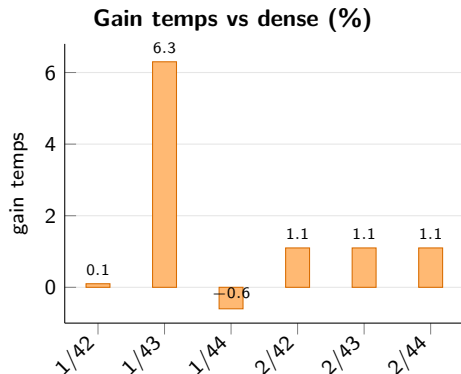
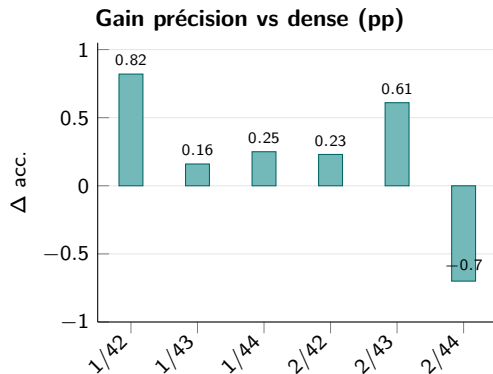
- ▶ `zen.univ-lille.fr`, exécution Slurm via `sbatch`
- ▶ $2 \times$ AMD EPYC 9354 : 64 cœurs physiques, 128 CPU logiques
- ▶ 2 domaines NUMA ; SIMD : FMA, AVX2, AVX-512
- ▶ caches (`lscpu`) : L1d/L1i 32 KB, L2 1 MB, L3 32 MB
- ▶ modules : `gcc/13.2.0`, `openblas/0.3.24`, `openmpi/4.1.6`

Ressources et protocole

- ▶ validation IST : 1 nœud, 2 ranks MPI, 1 CPU par rank
- ▶ sweep scaling : 1 nœud, jusqu'à 8 ranks MPI, 1 CPU par rank
- ▶ runs intra-nœud uniquement : pas de coût réseau inter-nœuds
- ▶ Tiny-ImageNet, batch 64, learning rate 0.01
- ▶ IST : $K \in \{1, 4\}$, seeds 42/43/44, runs de 1 à 2 epochs

Annexe : dense vs IST $K = 4$

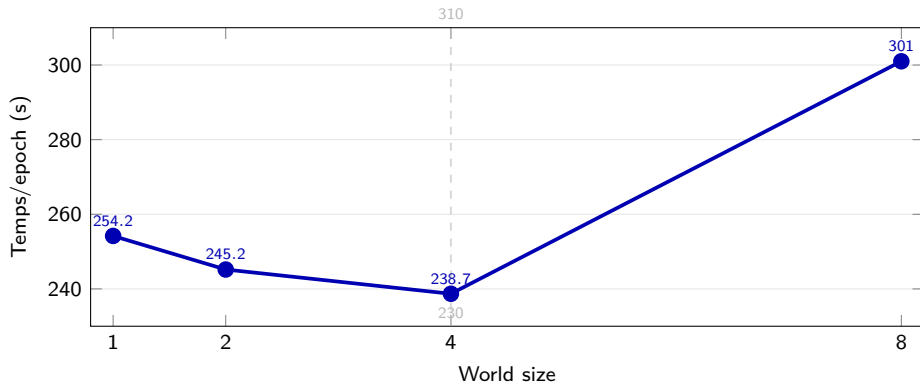
Synthèse multi-seed



À 2 epochs, le gain en temps se stabilise autour de +1.1%, tandis que le gain en précision reste dépendant du seed.

Annexe : sweep MPI rapide

Point de retournement en world size



Tiny-ImageNet, 1 epoch, batch 64, seed 42 : le gain est visible de $ws=1$ à $ws=4$, puis disparaît à $ws=8$.

- ▶ $ws=4$: meilleur point
- ▶ $ws=8$: dégradation nette
- ▶ batch global fixé, sweep intra-nœud